

Projet Final Industriel

SenLogix – Plateforme logistique distribuée

De la conception métier au déploiement Cloud-Native supervisé

Master Système d'information – SOA

Éditeur : Gana Faye

Contexte industriel

SenLogix interconnecte des boutiques e-commerce avec des compagnies de livraison express afin de créer, suivre et notifier les livraisons de colis.

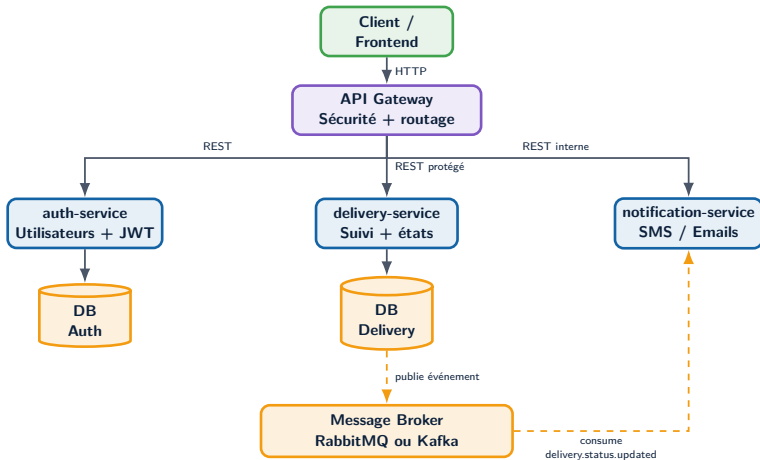
- ▶ architecture à microservices autonomes ;
- ▶ sécurité centralisée par API Gateway ;
- ▶ communication synchrone REST et asynchrone événementielle ;
- ▶ conteneurisation complète avec Docker Compose.
- ▶ observabilité avec Prometheus et Grafana ;
- ▶ bases de données isolées par service ;
- ▶ résilience via compensation métier ;
- ▶ livrable valorisable dans un portfolio d'ingénieur.

Ce projet synthétise l'ensemble du programme

L'étudiant doit démontrer sa capacité à concevoir, développer, sécuriser, déployer et superviser une application distribuée complète.

- ▶ Modéliser un domaine métier logistique en services autonomes.
- ▶ Définir des contrats d'API et des événements métier stables.
- ▶ Mettre en place JWT, routage et contrôle d'accès.
- ▶ Orchestrer l'architecture en local avec une seule commande.
- ▶ Mesurer la santé technique du système en conditions d'exploitation.

Architecture cible



Composants obligatoires

Composant	Rôle	Technologies possibles
auth-service	utilisateurs, rôles, JWT	FastAPI ou Spring Boot
delivery-service	création et suivi colis	FastAPI ou Spring Boot
notification-service	consumer événements	Python ou Node.js
API Gateway	sécurité et routage	Kong, Traefik, Nginx
Message Broker	événements métier	RabbitMQ ou Kafka
Observabilité	métriques et dashboard	Prometheus + Grafana

Rôle

Gérer les utilisateurs de la plateforme : clients, boutiques et livreurs.

- ▶ inscription ;
- ▶ connexion ;
- ▶ stockage des utilisateurs dans une base isolée ;
- ▶ génération de jetons JWT signés.

Claims JWT attendus

- ▶ `sub` : identifiant utilisateur ;
- ▶ `role` : `ROLE_BOUTIQUE` ou `ROLE_LIVREUR` ;
- ▶ `exp` : expiration du jeton ;
- ▶ `iat` : date d'émission.

Rôle métier

Cœur de SenLogix : il gère la création d'une demande de livraison et le cycle de vie du colis.

- ▶ adresse de collecte ;
- ▶ adresse de livraison ;
- ▶ poids du colis ;
- ▶ destinataire et contact ;
- ▶ affectation ou réservation d'un livreur.

Statuts minimaux

- ▶ EN_COURS_DE_COLLECTE ;
- ▶ AU_DEPOT ;
- ▶ EN_COURS_DE_LIVRAISON ;
- ▶ LIVRE ;
- ▶ ECHOUE si compensation.

Nom de l'événement

À chaque changement de statut critique, delivery-service publie `delivery.status.updated` dans le broker.

```
{
  "event_id": "evt-2026-0001",
  "event_type": "delivery.status.updated",
  "delivery_id": "del-1024",
  "status": "EN_COURS_DE_LIVRAISON",
  "recipient_name": "Awa Diop",
  "recipient_phone": "+221770000000",
  "occurred_at": "2026-05-21T18:00:00Z"
}
```

Service 3 : notification-service

Rôle

Alerter le destinataire final en temps réel lorsqu'un événement de livraison est reçu.

- ▶ Pas d'API HTTP publique.
- ▶ Pas de base de données obligatoire.
- ▶ Service exécuté comme worker d'arrière-plan.
- ▶ Consomme `delivery.status.updated` depuis RabbitMQ ou Kafka.
- ▶ Simule l'envoi SMS ou e-mail par un log applicatif.

Exemple de message simulé

Bonjour, votre colis est en cours de livraison par notre coursier.

API Gateway

- ▶ point d'entrée unique ;
- ▶ routage vers les services internes ;
- ▶ validation du JWT ;
- ▶ blocage des requêtes anonymes.

Règles attendues

- ▶ /auth/** public ;
- ▶ /deliveries/** protégé ;
- ▶ rôle boutique pour créer une livraison ;
- ▶ rôle livreur pour changer un statut.

Échéance 2 : Résilience et Saga

Scénario recommandé

Lors de la création d'une livraison, le système tente de réserver un livreur. Si aucun livreur n'est disponible, une compensation annule la demande.

Étape	Succès	Échec / compensation
Créer livraison	statut PENDING	–
Réserver livreur	statut ASSIGNED	aucun livreur
Compensation	–	statut ECHOUÉ
Notification	événement succès	événement échec

Échéance 3 : DevOps et conteneurisation

- ▶ Chaque microservice possède son propre Dockerfile optimisé.
- ▶ Les builds multi-stage sont attendus pour réduire la taille des images.
- ▶ Un `docker-compose.yml` unique lance toute l'infrastructure.
- ▶ Les bases de données sont séparées par service.
- ▶ Le broker, la gateway, Prometheus et Grafana sont inclus dans la stack.

Commande cible

```
docker compose up -build
```

Prometheus

- ▶ collecte `/metrics` ;
- ▶ vérifie les targets ;
- ▶ centralise les séries temporelles ;
- ▶ expose les requêtes PromQL.

Grafana

- ▶ requêtes HTTP par seconde ;
- ▶ taux d'erreurs 4xx et 5xx ;
- ▶ latence moyenne ou p95 ;
- ▶ messages en attente dans le broker.

Structure de dépôt recommandée

```
senlogix/  
  auth-service/  
    Dockerfile  
    src/  
  delivery-service/  
    Dockerfile  
    src/  
  notification-service/  
    Dockerfile  
    src/  
  gateway/  
    nginx.conf ou traefik.yml  
  observability/  
    prometheus.yml  
    grafana/  
  postman/  
    SenLogix.postman_collection.json  
  docker-compose.yml  
  README.md
```

1. Créer un compte boutique via `auth-service`.
2. Se connecter et récupérer un JWT signé.
3. Appeler la Gateway avec `Authorization: Bearer <token>`.
4. Créer une livraison dans `delivery-service`.
5. Modifier le statut du colis.
6. Vérifier l'événement dans le broker.
7. Observer le log de `notification-service`.
8. Contrôler les métriques dans Grafana.

1. Dépôt Git structuré

Monorepo propre contenant les sources, configurations, Dockerfiles et fichiers d'observabilité.

2. README.md d'ingénieur

Architecture, diagramme des flux, choix techniques, variables d'environnement et commande de lancement.

3. Collection Postman

Requêtes pré-configurées pour générer le token, créer une livraison et tester le flux complet en moins de cinq minutes.

Axe	Attendu	Poids
Fonctionnel	auth, livraison, notifications, événements	30%
Architecture	découpage services, contrats, bases isolées	20%
Sécurité	JWT, rôles, gateway, routes protégées	15%
DevOps	Dockerfiles, Compose, reproductibilité	15%
Observabilité	métriques, Prometheus, Grafana	10%
Qualité	README, Postman, clarté du code	10%

Proposition de découpage en sprints

Sprint 1 – Contrats et Auth

- ▶ structure du monorepo ;
- ▶ OpenAPI auth ;
- ▶ inscription, login, JWT.

Sprint 2 – Livraisons

- ▶ CRUD livraison ;
- ▶ statuts ;
- ▶ base isolée.

Sprint 3 – Broker et notifications

- ▶ publication événement ;
- ▶ consumer ;
- ▶ logs de notification.

Sprint 4 – Cloud-Native

- ▶ Gateway ;
- ▶ Docker Compose ;
- ▶ Prometheus + Grafana.

Première étape concrète

Initialiser la structure des dossiers et commencer le **TP 1 du projet final** : le premier service d'API avec son contrat d'interface OpenAPI.

- ▶ Créer le monorepo `senlogix/`.
- ▶ Définir les endpoints de `auth-service`.
- ▶ Documenter le contrat OpenAPI.
- ▶ Ajouter les premières requêtes Postman.
- ▶ Préparer le premier Dockerfile.

SenLogix

Concevoir, sécuriser, déployer, observer

Projet final industriel – Architecture SOA et Cloud-Native